MCPs na prática: conectando sua IA a qualquer ferramenta em Python

Asimov Academy

ASIMOV

Conteúdo

01. Introdução ao curso **4**
De onde surgiram os MCPs? . . . 4
O que vamos aprender? . . . 4

02. Como uma aplicação IA funciona? **6**
O que estamos chamando de “Aplicação IA”? . . . 6
Parte 1 - O Frontend . . . 7
Parte 2 - O Backend . . . 7
Parte 3 - A resposta . . . 8
Relembrando . . . 8

03. Agents e Tools - como uma aplicação IA acessa ferramentas **9**
A mesma divisão: Frontend e Backend . . . 9
O fluxo com tools (ferramentas) . . . 10
Por que isso é importante? . . . 11

04. Como criar Tools customizados **12**
Por que criar Tools customizados? . . . 12
1. LangChain (Python) . . . 13
2. Anthropic (Claude) . . . 13
3. OpenAI (Function Calling) . . . 13
4. Google Gemini (Function Calling) . . . 14
O grande desafio sem MCP . . . 14
Como o MCP ajuda aqui . . . 14

05. Modelo Context Protocol - como os MCPs padronizam o uso de Tools **15**
O desafio de tools customizadas . . . 15
Como o MCP resolve isso . . . 15
Vantagens principais . . . 16
Fluxo de uso simplificado . . . 16

06. Criando um servidor MCP local **17**
Dependências . . . 17
Código do primeiro servidor MCP . . . 17
Testando o servidor MCP . . . 18

07. Criando um servidor MCP remoto **19**

08. Miniprojeto - seu primeiro cliente de MCP para busca na Wikipedia **20**

09. O que de fato define um cliente de MCP? **22**
01. Descoberta dinâmica de ferramentas . . . 22
02. Decisão automática sobre qual tool usar . . . 22
03. Validação de dados e manutenção de contexto . . . 22
04. Interface para conversa contínua . . . 22
05. Gerenciamento de erros . . . 23
06. Exibição de ferramentas usadas . . . 23
Conclusão . . . 23

10. Explorando clientes de MCP **24**
5ire . . . 24
AIaW . . . 24
Chainlit . . . 24
Claude Desktop . . . 24
Cline . . . 24
Cursor . . . 25
Continue . . . 25
Dolphin-MCP . . . 25
Apify MCP Tester . . . 25
FLUJO . . . 25
Encerramento . . . 26

11. Conectando seu próprio MCP ao Claude Desktop **27**
Conectando o MCP local ao Claude . . . 27
O que foi escrito? . . . 28

12. Conectando MCPs externos ao Claude Desktop **29**
O que são os argumentos? . . . 29
E quanto a erros? . . . 29
De onde vem este servidor remoto? . . . 30
E quanto a outros servidores remotos? . . . 30

13. Conectando MCPs com CursorAI **31**
Conectando o MCP local ao CursorAI . . . 31
E quanto a erros? . . . 32
E quanto a outros servidores remotos? . . . 32

14. Miniprojeto - criando um MCP de previsão do tempo **34**

15. Integrando o MCP de previsão do tempo com todos os clientes **36**
Cliente “manual” (código Python) . . . 37
Claude Desktop . . . 37
CursorAI . . . 38

16. Fornecendo mais contexto aos seus tools **40**
A importância dos type hints e docstrings . . . 40
Exemplo prático . . . 40
Benefícios adicionais . . . 41
Conclusão . . . 41

17. Outros conceitos de MCPs **42**
Prompts: reutilizando instruções de forma padronizada . . . 42
Resources: fornecendo dados contextuais aos modelos . . . 42
Roots: delimitando o escopo de atuação dos servidores . . . 43
Sampling: solicitando completions de LLMs via cliente . . . 43
Conclusão . . . 43

18. Encerramento **44**
Recapitulação dos aprendizados . . . 44
Próximos passos . . . 44
Dicas para ir além . . . 44

01. Introdução ao curso

Se você já passou dificuldade pra integrar modelos de IA com ferramentas customizadas, **esse curso é pra você**.

Aqui, você vai aprender sobre **MCPs**, um conceito inovador dentro do mundo de IA, que tem gerado um bom hype nos últimos tempos. E claro, já tem muita gente falando todo tipo de besteira sobre o assunto…

Vocês sabem: nossa equipe na Asimov Academy é bem "pé no chão". Mesmo assim, estamos surpresos com a capacidade dos MCPs!

De onde surgiram os MCPs?

Hoje em dia, aplicações de IA vão muito além de chatbots. Elas se conectam a bancos de dados, APIs, planilhas, e até sistemas internos de empresas. E se não seguirmos um padrão, cada integração vira um projeto novo, que deve ser construído do zero.

O MCP - Model Context Protocol - surgiu para resolver exatamente isso. Ele padroniza a forma que a gente conecta grandes modelos de IA como ChatGPT, Gemini, Llama a qualquer fonte de dados ou ferramenta.

Mas esse não é um curso meramente conceitual! Aqui a gente vai botar a mão na prática e vai construir nossos próprios servidores MCP em tempo recorde.

Afinal, é só assim que vocês vão entender de fato onde que estão as vantagens de construir um sistema de IA, uma aplicação de IA que utiliza MCPs.

O que vamos aprender?

- Entender qual a arquitetura que se usa pra criar chatbots como o ChatGPT, e como ela foi evoluindo com o tempo.
- Nós vamos também tirar a cortina e explicar como agentes de IA escolhem utilizar uma ferramenta ou outra, e por que os MCPs estão sendo tão discutidos e relevantes nesse meio.
- Você vai entender a arquitetura dos MCPs, com a ideia de cliente e servidor trocando ferramentas entre eles.
- A gente vai também criar nossos próprios servidores MCP, que podem rodar qualquer código Python de forma local ou remota, podendo até ser hospedado em qualquer lugar.
- Você vai explorar o uso de clientes de MCP como Claude Desktop e CursorAI, mostrando como MCPs se conectam a eles - inclusive o nosso próprio servidor escrito em Python.

- E ao final, vamos montar um projeto prático de MCP que faz a previsão do tempo, pronto para se integrar com diferentes clientes.

02. Como uma aplicação IA funciona?

Nessa aula, vamos entender como funciona uma aplicação de IA simples — como o ChatGPT, por exemplo.

Você provavelmente já usou alguma ferramenta baseada em IA. Mas já parou pra pensar no que acontece por trás das cortinas quando você digita uma pergunta e a IA responde?

O que estamos chamando de "Aplicação IA"?

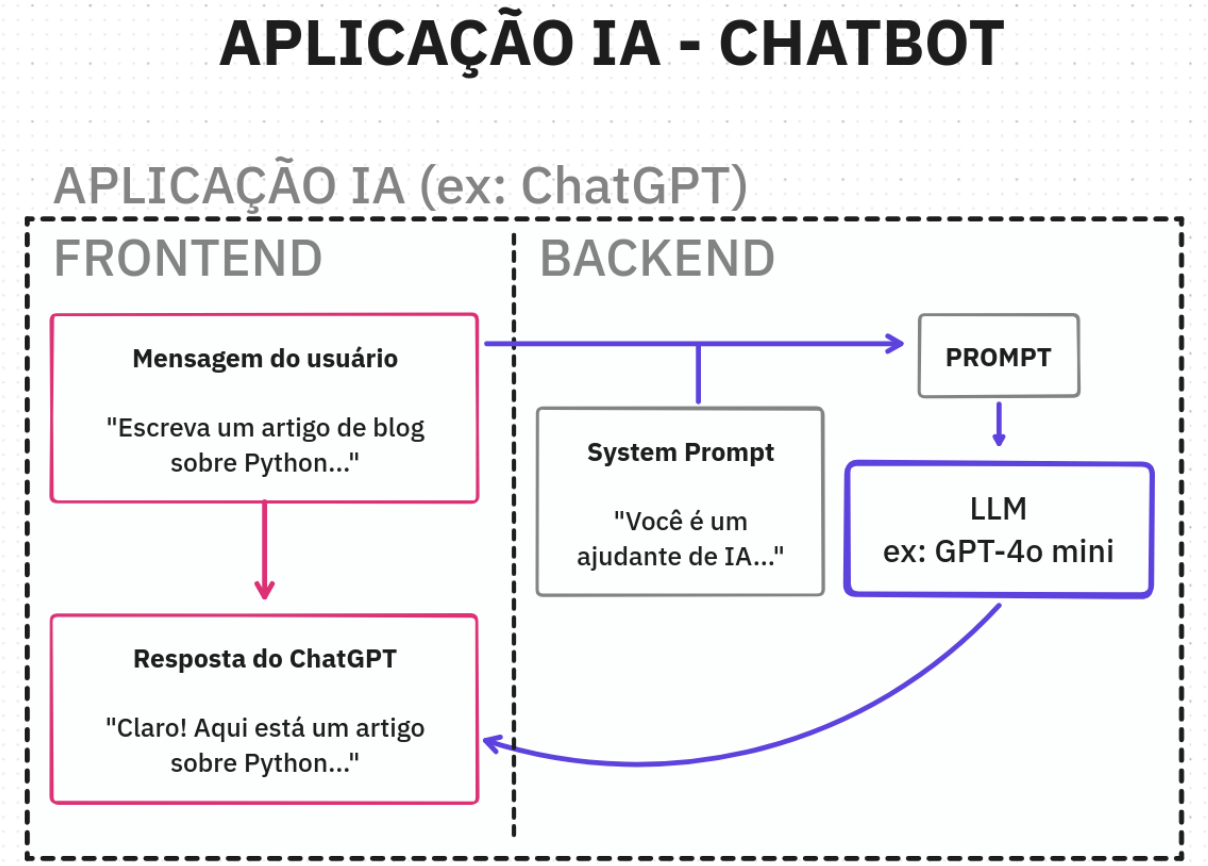

Figure 1: Fluxo de uma aplicação IA básica (chatbot)

Quando a gente fala em "aplicação IA", estamos falando de uma interface que conversa com um modelo de linguagem — uma LLM.

No nosso exemplo, vamos imaginar algo como o ChatGPT. É uma aplicação relativamente simples do ponto de vista estrutural.

Pra facilitar, vou dividir essa aplicação em duas partes: **frontend** e **backend**.

Parte 1 - O Frontend

O frontend é o que o usuário vê.

É a interface onde a pessoa digita sua pergunta. Por exemplo:

```
"Escreva um artigo de blog sobre Python…"
```

Essa mensagem é enviada para o backend. E depois, o usuário recebe a resposta da IA, como:

```
"Claro! Aqui está um artigo sobre Python…"
```

Essa parte da conversa é simples. Mas a mágica acontece do lado de dentro, no backend. Vamos pra lá?

Parte 2 - O Backend

No backend, o papel principal é preparar o **prompt final** que será enviado ao modelo.

Esse prompt final geralmente é formado por duas partes principais:

1. **O prompt do usuário** (o que ele digitou)
2. **Um ou mais "system prompts"** — instruções que definem o comportamento da IA.

Um system prompt é algo como:

```
"Você é um assistente de IA. Responda de forma clara e objetiva…"
```

Essa instrução serve pra guiar o comportamento do modelo. É como definir o "papel" que a IA vai interpretar.

O backend junta tudo isso e monta o prompt final. Esse prompt é então enviado para o modelo de linguagem — por exemplo, o **GPT-4o mini**, no nosso caso.

Parte 3 - A resposta

Depois que o modelo processa o prompt, ele gera uma resposta e envia de volta pro backend. Essa resposta, por fim, aparece lá na interface — no frontend — pra que o usuário possa ler.

Então o fluxo completo fica assim:

1. Usuário escreve uma mensagem.
2. Backend junta essa mensagem com o system prompt.
3. Modelo processa e responde.
4. Resposta aparece na tela.

Simples, né?

Relembrando

Esse tipo de aplicação é o que estamos chamando de **aplicação IA "básica"** — ou seja, uma aplicação que não interage com nenhuma ferramenta externa.

Ela é só uma ponte entre o usuário e um modelo de linguagem.

Mas como você vai ver nas próximas aulas, dá pra ir muito além disso.

Na próxima aula, vamos explorar o que acontece quando a IA precisa **usar ferramentas externas** — como buscar informações na internet, acessar arquivos, rodar código, e por aí vai.

Isso vai abrir caminho para entendermos os **agents**, as **tools**, e finalmente os **MCPs**.

03. Agents e Tools - como uma aplicação IA acessa ferramentas

Na aula passada, você viu como funciona uma aplicação IA simples, como o ChatGPT. Agora, vamos dar um passo além.

Imagine que você faz a seguinte pergunta pra IA:

```
"Quem é o presidente dos Estados Unidos?"
```

Se o modelo tiver essa resposta na sua "memória", ele responde direto. Mas… e se a informação estiver desatualizada? E se a IA quiser confirmar antes de responder?

É aí que entram as **ferramentas externas** — e é aqui que a aplicação vira um **Agent**.

Vamos entender como isso funciona por dentro.

A mesma divisão: Frontend e Backend

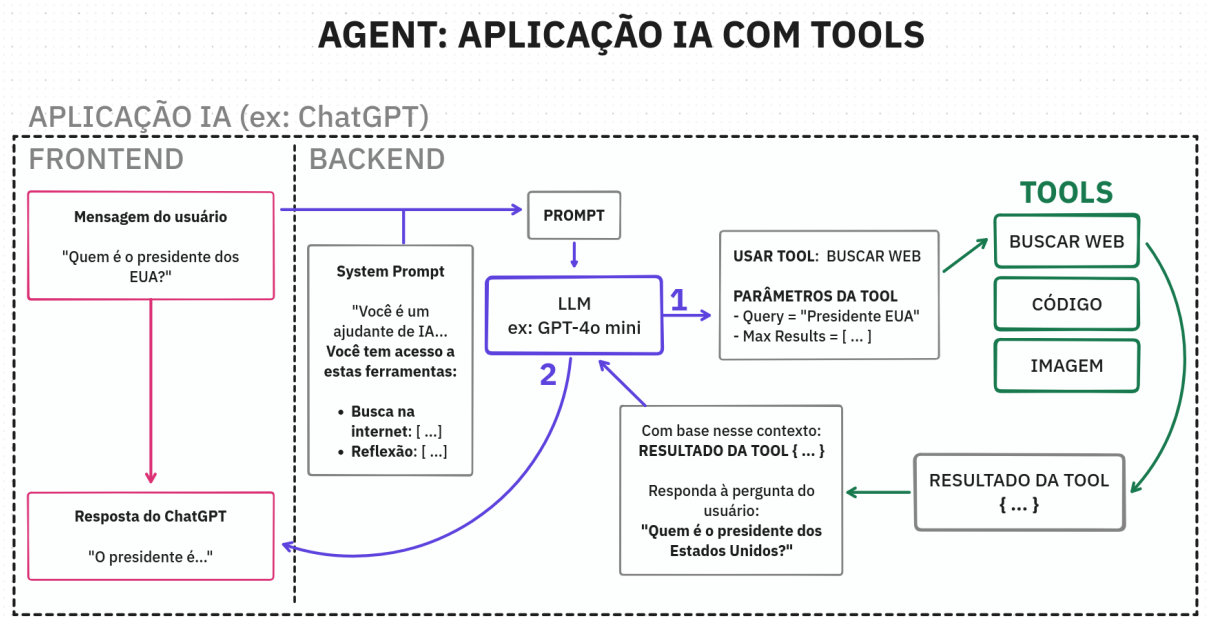

Figure 2: Fluxo de uma aplicação IA com Agent e Tools

Assim como na aula anterior, temos duas partes principais:

- O **frontend** continua sendo a interface onde o usuário digita a pergunta e recebe a resposta.
- O **backend** é onde tudo acontece: o modelo é acionado, as ferramentas são chamadas, e a resposta final é preparada.

Mas agora, no backend, o modelo **pode tomar decisões**. Ele pode, por exemplo, perceber que **não sabe** a resposta e decidir usar uma ferramenta externa, como uma busca na web.

O fluxo com tools (ferramentas)

Vamos ver o que muda.

O usuário envia a pergunta:

```
"Quem é o presidente dos Estados Unidos?"
```

Essa pergunta, junto com instruções prévias (os system prompts), é enviada ao modelo. Mas agora, o prompt avisa:

```
"Você tem acesso às seguintes ferramentas…"
```

Essas ferramentas podem ser:

- Web Search (busca online)
- Gerador de imagens
- Cálculos avançados
- Acesso a banco de dados
- Entre outras

Ao analisar a pergunta, o modelo entende que **precisa buscar na internet** pra garantir uma resposta atualizada.

Então, o modelo **chama a ferramenta de busca**, usando parâmetros como:

- `query = "Presidente EUA"`
- `max_results = 3`

A ferramenta faz a busca, coleta os resultados e envia de volta ao modelo.

Agora sim, com base nesse novo contexto (os resultados da busca), o modelo elabora uma resposta.

Essa resposta é enviada para o frontend e exibida ao usuário:

```
"O presidente dos Estados Unidos é ..."
```

Tudo isso acontece em segundos, de forma transparente pra quem está do outro lado da tela.

Por que isso é importante?

Esse tipo de arquitetura é o que torna a IA **muito mais poderosa**. Com acesso a ferramentas, o modelo deixa de depender só do que aprendeu no treinamento e passa a **interagir com o mundo real em tempo real**.

Ele pode consultar, calcular, pesquisar, gerar imagens e muito mais.

E o mais interessante: **ele decide sozinho quando usar cada ferramenta**.

É isso que diferencia uma aplicação IA simples de uma aplicação com Agent.

Na próxima aula, a gente vai entender como podemos enviar nossas próprias tools para o modelo.

04. Como criar Tools customizados

Nesta aula, vamos ver como você, desenvolvedor, cria suas próprias tools ou chamadas de função para IA. Cada biblioteca ou provedor de LLM tem sua maneira específica de definir e usar essas ferramentas. O objetivo aqui é entender as diferenças e mostrar por que isso costuma demandar código "do zero" toda vez.

Por que criar Tools customizados?

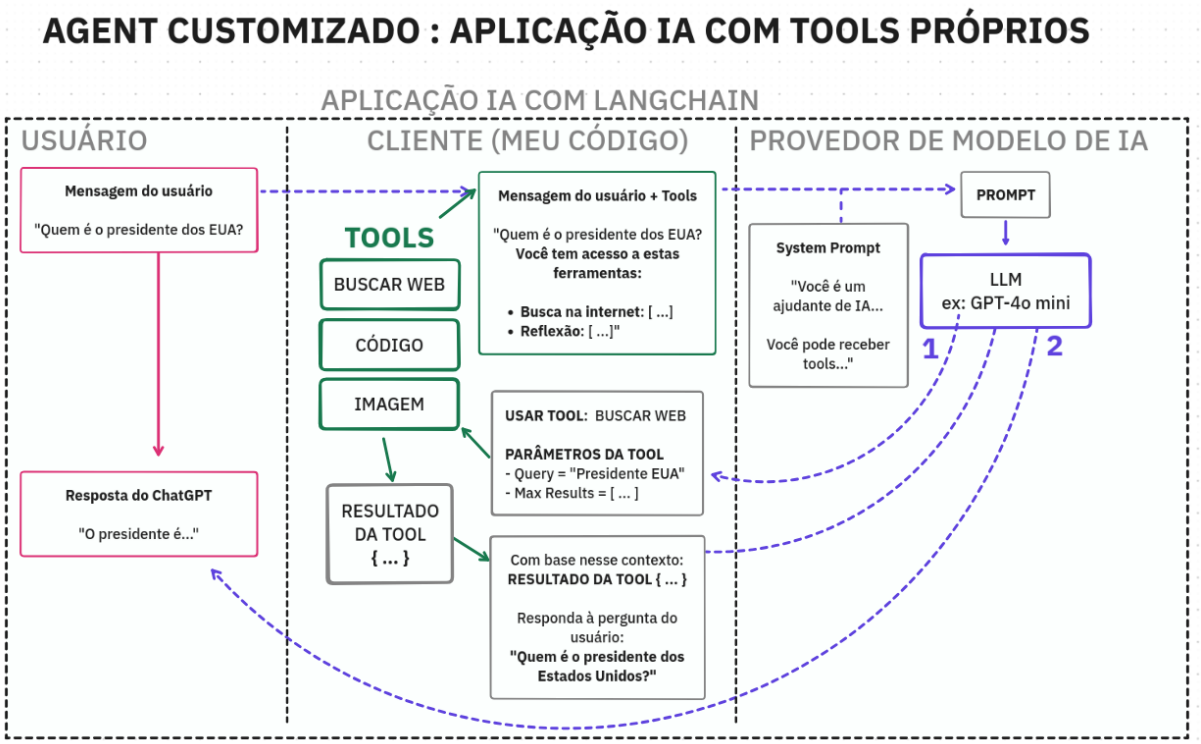

Figure 3: Fluxo de uma aplicação IA com Tools próprios

Você pode querer que sua IA:

1. Acesse seu repositório no GitHub
2. Leia e escreva arquivos no Google Drive
3. Busque anotações no Notion
4. Interaja com sistemas internos da empresa

Para cada caso, você vai precisar "ensinar" à IA como chamar aquela ferramenta. E isso envolve:

- Definir a interface (nome, parâmetros, retorno)
- Escrever o código que efetivamente faz a chamada
- Lidar com erros, autenticação e formatos de resposta

Cada LLM ou framework faz isso de um jeito. Vamos ver quatro exemplos comuns.

1. LangChain (Python)

Na LangChain, você define uma classe ou função que implementa a interface `Tool`.

- Declare `name`, `description` e o método que executa a ação
- Registre essa tool num catálogo que o Agent lê
- Ao chamar o Agent, as tools ficam disponíveis automaticamente

Exemplo simplificado em Python:

```
from langchain.tools import BaseTool

class GitHubTool(BaseTool):
    name = "github_prs"
    description = "Busca pull requests abertas num repositório"

    def _run(self, repo: str):
        # aqui vai o código que usa a API do GitHub
        return lista_de_prs
```

2. Anthropic (Claude)

No modelo Claude, a Anthropic oferece um “Tool Use API” em que você:

- Define um JSON com `tool_name`, `arguments_schema`, etc.
- Passa isso no prompt do system ou nas opções da chamada
- A IA retorna uma estrutura indicando qual tool usar

A diferença principal é que tudo roda via JSON no chat. Você não cria classes, mas monta objetos de configuração para cada ferramenta.

3. OpenAI (Function Calling)

A API de function calling da OpenAI é baseada em JSON Schemas. Para cada função você:

- Especifica nome, descrição e esquema dos parâmetros
- Informa essas funções na chamada de chat, em `functions=[…]`

- Se o modelo decidir chamar sua função, ele devolve um JSON com `name` e `arguments`

Exemplo de definição de função:

```
{
  "name": "get_drive_file",
  "description": "Busca arquivo no Google Drive",
  "parameters": {
    "type": "object",
    "properties": {
      "file_id": { "type": "string" }
    },
    "required": ["file_id"]
  }
}
```

4. Google Gemini (Function Calling)

O Gemini segue padrão parecido com OpenAI, mas com algumas diferenças:

- Usa o campo `@context` para registrar as funções
- Os exemplos oficiais mostram como chamar reuniões, acessar contatos, etc.
- A forma de lidar com o retorno e erros pode variar um pouco

Cada provedor tem seu "dialeto" de JSON e regras de uso.

O grande desafio sem MCP

- Toda vez que você muda de LangChain para OpenAI ou Claude, precisa reescrever suas tools
- Você acaba mantendo vários projetos parecidos, mas com APIs e formatos diferentes
- Isso toma tempo e gera inconsistências

Como o MCP ajuda aqui

Com o **Model Context Protocol** você define um servidor de tools padronizado:

- Um mesmo serviço HTTP (ou stdio/SSE) expõe as ferramentas
- Os clientes (LangChain, Claude Desktop, CursorAI…) falam a mesma linguagem
- Você implementa cada tool uma única vez, no servidor MCP
- Trocar de cliente não exige reescrever suas tools

Na próxima aula, faremos um miniprojeto prático criando um servidor MCP de previsão do tempo. Assim, você vai ver na prática como criar tools uma única vez e usá-las em qualquer cliente. Te espero lá!

05. Modelo Context Protocol - como os MCPs padronizam o uso de Tools

Nesta aula, vamos entender por que o MCP é um divisor de águas na forma como conectamos LLMs a ferramentas externas. Você vai ver o que passa a ser possível quando padronizamos essas integrações.

O desafio de tools customizadas

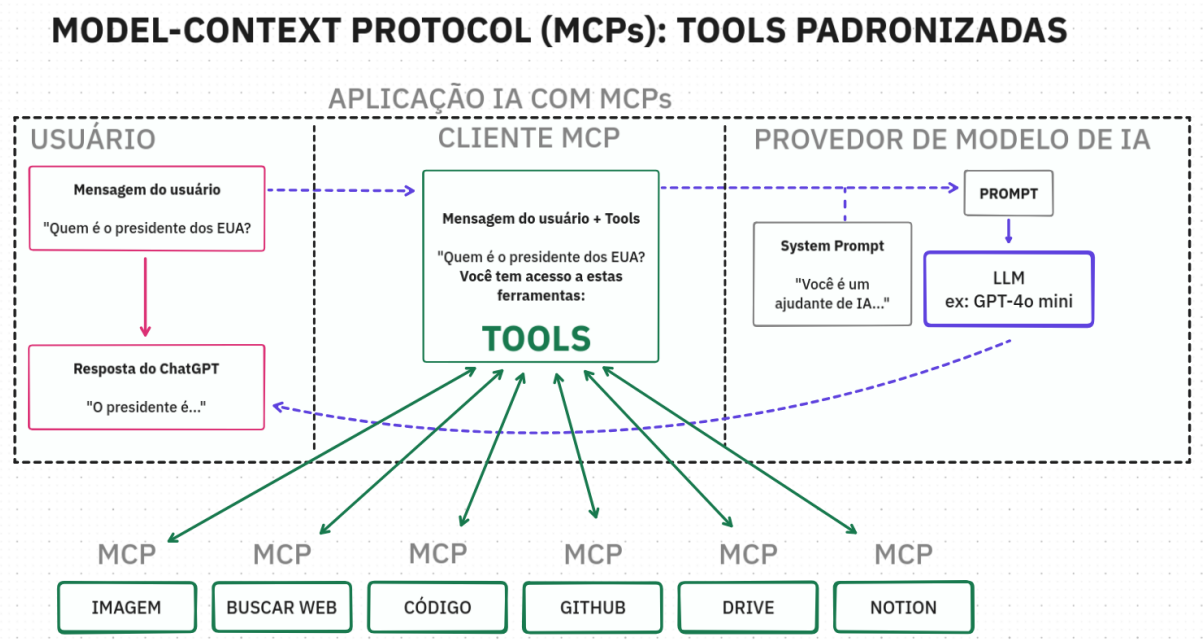

Figure 4: Fluxo de uma aplicação IA com MCPs

Hoje, para cada LLM ou framework você precisa:

- Escrever do zero a definição da tool
- Tratar autenticação, erros e formatos de resposta
- Adaptar o JSON ou a classe ao "dialeto" do provedor (LangChain, OpenAI, Anthropic, Google…)

Isso gera muita duplicação de código e retrabalho sempre que você muda de provedor.

Como o MCP resolve isso

O MCP define um **protocolo único** para:

1. **Listar** quais tools existem
2. **Executar** uma tool específica
3. **Retornar** os resultados

Sem se preocupar com a biblioteca ou LLM usada pelo cliente. MCP é como um "USB-C para IA": padrão, estável e interoperável modelcontextprotocol.io.

Vantagens principais

Isolamento cliente/servidor O cliente (seja Python, Claude Desktop ou CursorAI) não precisa saber o código interno da tool. Ele só envia um pedido e recebe um resultado.

Suporte a IA fechada Você pode expor funcionalidades para modelos que você não controla diretamente (por exemplo, Claude ou Gemini), porque o cliente MCP fala o mesmo protocolo para todos.

Ecossistema de "plugins" Cada servidor MCP funciona como um plugin independente. Basta subir o serviço em qualquer servidor ou rodá-lo localmente, e o cliente passa a ter acesso à nova ferramenta.

Colaboração e extensibilidade Outros desenvolvedores podem criar MCPs específicos (GitHub, IMDB, Google Maps, sistemas internos da empresa…) e compartilhar com você. Não é preciso reinventar a roda.

Fluxo de uso simplificado

1. **Descoberta**: Cliente MCP pergunta ao servidor: "Quais tools você oferece?"
2. **Requisição**: Cliente MCP envia: "Para a ação X, preciso da tool Y. Servidor, me mande a tool."
3. **Resposta**: Servidor MCP retorna o código da tool
4. **Execução**: Cliente MCP executa o código da tool

Com isso, a aplicação IA com ferramentas (Agent) fica extremamente modular e flexível. É possível desenvolver MCPs para qualquer plataforma:

- **MCP Google Maps**
- **MCP GitHub**
- **MCP IMDB**

Basta apontar seu cliente para esses servidores e começar a usar, sem escrever uma linha de integração nova para cada ambiente.

06. Criando um servidor MCP local

Agora, vamos criar nosso primeiro servidor MCP com Python.

Também iremos testar esse servidor usando um cliente mínimo (muito menos interessante que os clientes que usaremos mais para a frente).

Dependências

Para isso, usaremos a biblioteca FastMCP. Podemos instalá-la com:

```
pip install fastmcp
```

Mas o ideal é usar o `uv`, um gerenciador de pacotes de Python mais moderno e recomendado pelo `fastmcp`: Link de acesso ao uv

```
uv init
uv add fastmcp
```

O uv se encarrega de criar um ambiente virtual automaticamente depois de dar o `uv init`.

De qualquer forma, é preciso ter o pacote `fastmcp` instalado no seu ambiente!

Código do primeiro servidor MCP

O código abaixo utiliza o pacote `fastmcp` para criar um servidor MCP local em Python:

`servidor.py`

```python
from fastmcp import FastMCP

servidor_mcp = FastMCP('mcp-teste')


@servidor_mcp.tool()
async def dar_bom_dia(nome_usuario: str, id_usuario: int) -> str:
    return f"Olá, {nome_usuario}! (ID: {id_usuario})!"


if __name__ == "__main__":
    servidor_mcp.run(transport='stdio')
```

Note que:

- O `stdio` significa que o servidor MCP roda diretamente da sua máquina local!
- Precisamos definir a função com `async` para que ela rode de forma assíncrona por padrão (o pacote `fastmcp` requer isso).

Testando o servidor MCP

cliente.py

```
import asyncio
from pathlib import Path

from fastmcp import Client

caminho_servidor = Path(__file__).parent / "servidor.py"
cliente_mcp = Client(caminho_servidor)


async def testar_servidor(cliente, nome_usuario, id_usuario):
    async with cliente:
        argumentos = {'nome_usuario': nome_usuario, "id_usuario": id_usuario}
        resultado = await cliente.call_tool("dar_bom_dia", arguments=argumentos)
        print(f'Resultado obtido do servidor MCP: {resultado}')


if __name__ == '__main__':
    # testar_servidor(cliente=cliente_mcp, nome_usuario="Juliano", id_usuario=5)  # Precisa ser
     ↪  async!
    asyncio.run(testar_servidor(cliente=cliente_mcp, nome_usuario="Juliano", id_usuario=5))
```

Aqui, apenas testamos que o código está funcionando - o servidor está “comunicável” a partir do cliente.

Como estamos rodando de forma local, o caminho do servidor é um arquivo.

07. Criando um servidor MCP remoto

Apenas fazendo pequenos ajustes, passamos de um servidor local (`stdio`) para um remoto (`sse`)!

`servidor.py`

```
from fastmcp import FastMCP

servidor_mcp = FastMCP('mcp-teste')


@servidor_mcp.tool()
async def dar_bom_dia(nome_usuario: str, id_usuario: int) -> str:
    return f"Olá, {nome_usuario}! (ID: {id_usuario})!"


if __name__ == "__main__":
    servidor_mcp.run(transport='sse')
```

`cliente.py`

```
import asyncio

from fastmcp import Client

caminho_servidor = 'http://localhost:8000/sse'
cliente_mcp = Client(caminho_servidor)


async def testar_servidor(cliente, nome_usuario, id_usuario):
    async with cliente:
        argumentos = {'nome_usuario': nome_usuario, "id_usuario": id_usuario}
        resultado = await cliente.call_tool("dar_bom_dia", arguments=argumentos)
        print(f'Resultado obtido do servidor MCP: {resultado}')


if __name__ == '__main__':
    asyncio.run(testar_servidor(cliente=cliente_mcp, nome_usuario="Juliano", id_usuario=5))
```

Note que o caminho para o servidor se tornou um link com `localhost`. Em tese, poderia ser qualquer servidor / endereço de IP no mundo!

08. Miniprojeto - seu primeiro cliente de MCP para busca na Wikipedia

Agora vamos entender como funcionaria um cliente MCP de fato - integrado com IA para formatar a resposta final.

Para isso, vamos desenvolver um MCP de **busca na Wikipedia** por um tópico qualquer.

O primeiro passo é adicionar o pacote `wikipedia` ao projeto:

```
# Usando pip
pip install wikipedia

# Usando uv
uv add wikipedia
```

Usaremos o ChatGPT para gerar a resposta final a partir da busca. Por isso, vamos também adicionar a biblioteca `openai`.

Além disso, usaremos o `python-dotenv` para guardar a chave de API da OpenAI em um arquivo `.env` seguro:

```
# Usando pip
pip install openai python-dotenv

# Usando uv
uv add openai python-dotenv
```

Crie um arquivo `.env` e adicione a chave de API lá dentro:

```
CHAVE_API_OPENAI="sua-chave-de-api-vai-aqui"
```

Agora, vamos ao código:

`servidor.py`

```
from fastmcp import FastMCP

import wikipedia

servidor_mcp = FastMCP('mcp-busca-wikipedia')


@servidor_mcp.tool()
async def buscar_wikipedia(busca: str) -> str:
    return wikipedia.summary(busca)


if __name__ == "__main__":
    servidor_mcp.run(transport='sse')
```

`cliente.py`

```python
import asyncio
import os

import dotenv
from fastmcp import Client
from openai import OpenAI

caminho_servidor = 'http://localhost:8000/sse'
cliente_mcp = Client(caminho_servidor)


async def testar_servidor(cliente, busca):
    dotenv.load_dotenv()
    api_key = os.environ['CHAVE_API_OPENAI']
    async with cliente:
        resultado = await cliente.call_tool("buscar_wikipedia", arguments={'busca': busca})
        print(resultado)
        mensagem_sistema = f"""
        Você é um bot que faz buscas no wikipedia e sintetiza a resposta.
        O usuário buscou pelo seguinte tema: "{busca}".
        Para esta busca, você recebeu a seguinte resposta: "{resultado}".
        Com base nesse conteúdo, formate uma resposta amigável ao usuário.
        """
        client = OpenAI(api_key=api_key)
        response = client.responses.create(
            model="gpt-4o-mini",
            instructions=mensagem_sistema,
            input="Pode me falar mais sobre este assunto?",
        )
        print(response.output_text)


if __name__ == '__main__':
    busca = 'Isaac Asimov'
    asyncio.run(testar_servidor(cliente=cliente_mcp, busca=busca))
```

09. O que de fato define um cliente de MCP?

Nesta aula, vamos explorar em detalhes quais capacidades um cliente MCP precisa oferecer para funcionar de forma completa e autônoma, seguindo o protocolo Model Context Protocol.

01. Descoberta dinâmica de ferramentas

Um cliente MCP deve ser capaz de **listar todas as tools disponíveis** no servidor sempre que iniciar ou a qualquer momento durante a execução.

Isso acontece por meio de uma chamada como `session.list_tools()`, que retorna em tempo real as definições de cada ferramenta, sem nenhuma configuração manual adicional.

02. Decisão automática sobre qual tool usar

Depois de descobrir as tools, o cliente precisa **decidir sozinho** quando e qual ferramenta invocar, com base nas instruções do LLM ou em regras de negócio.

Na prática, isso significa que o cliente passa ao LLM a lista de tools e seus esquemas (por exemplo, `input_schema`) e, quando o LLM sinaliza um uso de tool, o cliente faz a chamada específica.

03. Validação de dados e manutenção de contexto

Antes de chamar uma ferramenta, o cliente deve **validar** se os parâmetros fornecidos estão no formato correto, usando o esquema JSON de cada tool.

Além disso, precisa **manter o contexto** das conversas e resultados de chamadas anteriores para permitir múltiplas interações encadeadas sem perder informações.

04. Interface para conversa contínua

Um cliente completo oferece uma **interface interativa** (por exemplo, linha de comando, GUI ou API web) que aceita queries do usuário, exibe as respostas do LLM e gerencia o fluxo de mensagens.

Em Python, isso costuma ser feito com um loop de input/output, como em `while True: input() → process_query() → print()`, garantindo que o usuário possa conversar indefinidamente até optar por sair.

05. Gerenciamento de erros

Ferramentas podem falhar por diversos motivos (timeout, autenticação, formato inválido, etc.). O cliente deve **capturar exceções**, apresentar mensagens amigáveis e, quando possível, permitir novas tentativas ou fallback para outras tools.

Um padrão comum é envolver cada chamada de tool em blocos `try/except`, registrando logs detalhados e mantendo a sessão ativa mesmo após erros pontuais.

06. Exibição de ferramentas usadas

Para fins de **transparência e auditoria**, o cliente deve mostrar ao usuário quais tools foram invocadas em cada etapa da conversa, com seus parâmetros e resultados.

Isso facilita o entendimento de como o LLM chegou à aquela resposta e ajuda a diagnosticar problemas de execução ou de qualidade dos dados retornados.

Conclusão

Um cliente MCP eficaz deve:

- Descobrir e listar ferramentas disponíveis dinamicamente.
- Permitir a seleção explícita de ferramentas pelo modelo de linguagem.
- Validar dados e manter o contexto da conversa.
- Oferecer uma interface para interações contínuas e naturais.
- Gerenciar erros de forma clara e informativa.
- Registrar e exibir as ferramentas utilizadas para transparência.

Na próxima aula, veremos diversos exemplos de clientes MCP disponíveis e analisaremos como eles implementam essas capacidades de maneiras ligeiramente diferentes.

10. Explorando clientes de MCP

Nesta aula, vamos conhecer alguns clientes MCP populares, ver onde encontrá-los e entender rapidamente o que cada um oferece. A lista oficial no site do MCP inclui dezenas de implementações, e aqui vamos destacar alguns com diferentes perfis: desde editores de código até frameworks completos de agentes.

5ire

Site: https://5ire.app/

5ire é um assistente de IA de desktop multiplataforma com suporte a MCP, fácil de configurar e ideal para quem está começando.

AIaW

Site: https://aiaw.app/

AIaW é um cliente web leve e full-featured que oferece chat com MCP e roda diretamente no navegador.

Chainlit

Site: https://chainlit.io

Chainlit é um framework Python para construir aplicações conversacionais, agora com suporte nativo a MCP para integrar ferramentas e dados externos.

Claude Desktop

Site: https://claude.ai/download

A aplicação desktop da Anthropic traz o poder do Claude com integração plena a servidores MCP e acesso a ferramentas locais.

Cline

Site: https://github.com/cline/cline

Cline é uma extensão para VS Code que adiciona capacidades de agente ao editor, suportando MCP para criar e usar ferramentas em workflows de codificação.

Cursor

Site: https://cursor.com

Cursor é um editor de código AI-first que suporta MCP, permitindo navegar e editar seu código com ferramentas customizadas.

Continue

Site: https://continue.dev

Continue é um assistente de código open-source com integração a MCP, disponível como extensão para VS Code e JetBrains.

Dolphin-MCP

Site: https://github.com/cognitivecomputations/dolphin-mcp

Dolphin-MCP é um cliente de linha de comando e biblioteca Python que conecta qualquer LLM a servidores MCP via stdio e SSE.

Apify MCP Tester

Site: https://apify.com (Example Clients - Model Context Protocol)

Apify MCP Tester é um cliente JavaScript standalone para testar servidores MCP via SSE, sem necessidade de configuração prévia.

FLUJO

Site: https://flujo.orchestraight.co

FLUJO é uma interface de construção de workflows que integra diversas ferramentas via MCP, combinando automação no estilo n8n com chat de IA. O projeto é open-source e gratuito, ideal para prototipagem rápida de agentes IA com plugins padronizados.

Encerramento

Esses clientes representam apenas uma parte do ecossistema. Você encontrará muitas outras na internet!

Descubra mais em:

- https://modelcontextprotocol.io/clients
- https://github.com/punkpeye/awesome-mcp-clients

11. Conectando seu próprio MCP ao Claude Desktop

Vamos adaptar o servidor MCP de busca na Wikipedia para permitir seu uso no Claude Desktop:

`servidor_claude_wikipedia.py`

```
from fastmcp import FastMCP

import wikipedia

servidor_mcp = FastMCP('mcp-busca-wikipedia')


@servidor_mcp.tool()
async def buscar_wikipedia_local(busca: str) -> str:
    return wikipedia.summary(busca)


if __name__ == "__main__":
    servidor_mcp.run(transport='stdio')
```

Conectando o MCP local ao Claude

Agora, conseguimos conectar com o Claude Desktop editando o arquivo JSON de configuração, chamado `claude_desktop_config.json`:

`claude_desktop_config.json`

```
{
  "mcpServers": {
    "busca-wikipedia": {
      "command": "/home/USER/.local/bin/uv",
      "args": [
        "--directory",
        "/home/USER/CAMINHO_PASTA",
        "run",
        "servidor_claude_wikipedia.py"
    ],
    "description": "Um servidor MCP local que faz buscas na wikipedia e retorna o resumo de um
     ↪  artigo."
    }
  }
}
```

Por padrão, este arquivo fica no seguinte caminho:

- `%APPDATA%/Claude/config.json.` em Windows
- `~/Library/Application\  Support/Claude/claude_desktop_config.json` em Mac

- `~/.config/Claude/` em Linux

Mas também é possível acessá-lo a partir das configurações do Claude Desktop, indo no menu e selecionando `Arquivo` > `Configurações` > `Desenvolvedor` > `Editar configuração`.

O que foi escrito?

Note que o comando usado é o comando do `uv`. Para descobrir o caminho correto, use:

- `where uv`, em Windows
- `which uv`, em Mac/Linux

Além disso, foi preciso passar a **pasta até o arquivo** do servidor e o **nome do arquivo** nos argumentos.

Lembre-se de ajustar estes valores para o seu computador!

12. Conectando MCPs externos ao Claude Desktop

Vamos agora adicionar o MCP chamado `filesystem`. Este MCP permite que o Claude Desktop interaja com seus arquivos em pastas específicas.

Modifique seu arquivo JSON para o seguinte:

`claude_desktop_config.json`

```
{
  "mcpServers": {
    "busca-wikipedia": {
      "command": "/home/USER/.local/bin/uv",
      "args": [
        "--directory",
        "/home/USER/CAMINHO_PASTA",
        "run",
        "servidor_claude_wikipedia.py"
      ],
      "description": "Um servidor MCP local que faz buscas na wikipedia e retorna o resumo de um
      ↪  artigo."
    },
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/home/USER/Desktop",
        "/home/USER/Downloads"
      ]
    }
  }
}
```

O que são os argumentos?

Os argumentos indicam a quais pastas o MCP tem acesso. No caso acima, foi dado acesso às pastas de Desktop e Download.

Lembre-se de ajustar os valores para o seu computador!

E quanto a erros?

Se houver erros na configuração, confira:

- Os caminhos usados no arquivo `claude_desktop_config.json`
- Se o `uv` tem acesso a um ambiente virtual com as bibliotecas necessárias, na pasta do argumento `--directory`

- Se há espaços em branco no nome da pasta (em alguns casos pode ser incompatível com o Claude Desktop)

Tente também fechar e abrir o Claude Desktop algumas vezes para confirmar que os caminhos estão atualizados.

De onde vem este servidor remoto?

Note que aqui, não usamos um servidor remoto com `http://` `...`. No lugar disso, o Claude utiliza uma sintaxe própria com `npx`, que é um programa de linha de comando para rodar um código feito em Node (biblioteca de Javascript).

Se você não tiver o Node instalado, é preciso baixá-lo nesse link daqui.

E quanto a outros servidores remotos?

No momento, o Claude Desktop não aceita "qualquer" servidor MCP remoto, apenas os servidores pré-aprovados pela Anthropic! Isso é uma limitação/decisão da empresa. Os servidores oficiais estão listados aqui:

- Documentação
- GitHub

13. Conectando MCPs com CursorAI

Vamos adaptar o servidor MCP de busca na Wikipedia para permitir seu uso no CursorAI.

Aqui, já vamos criar duas versões do servidor - uma com conexão local (`stdio`), e outra remota (`sse`):

`servidor_cursorai_wikipedia_stdio.py`

```
from fastmcp import FastMCP

import wikipedia

servidor_mcp = FastMCP('mcp-busca-wikipedia')


@servidor_mcp.tool()
async def buscar_wikipedia_local(busca: str) -> str:
    return wikipedia.summary(busca)


if __name__ == "__main__":
    servidor_mcp.run(transport='stdio')
```

`servidor_cursorai_wikipedia_sse.py`

```
from fastmcp import FastMCP

import wikipedia

servidor_mcp = FastMCP('mcp-busca-wikipedia')


@servidor_mcp.tool()
async def buscar_wikipedia_remoto(busca: str) -> str:
    return wikipedia.summary(busca)


if __name__ == "__main__":
    servidor_mcp.run(transport='sse')
```

Conectando o MCP local ao CursorAI

O CursorAI possui uma configuração muito parecida com o Claude Desktop. Temos apenas que abrir o mesmo arquivo JSON de configuração, equivalente ao que usamos na aula anterior.

No caso do cursor, este arquivo se chama `mcp.json`:

`mcp.json`

```
{
  "mcpServers": {
    "busca-wikipedia-stdio": {
      "command": "/home/USER/.local/bin/uv",
      "args": [
        "--directory",
        "/home/USER/CAMINHO_PASTA",
        "run",
        "servidor_cursorai_wikipedia_stdio.py"
      ],
      "description": "Um servidor de MCP local que faz uma busca na Wikipedia e retorna um
       ↪  resumo do principal resultado."
    },
    "busca-wikipedia-sse": {
      "url": "http://localhost:8000/sse",
      "description": "Um servidor de MCP remoto que faz uma busca na Wikipedia e retorna um
       ↪  resumo do principal resultado."
    }
  }
}
```

Para acessar este arquivo, vá na configuração de MCP do CursorAI e clique em "Adicionar novo servidor MCP Global".

E quanto a erros?

Se houver erros na configuração, confira:

- Os caminhos usados no arquivo `mcp.json`
- Se o `uv` tem acesso a um ambiente virtual com as bibliotecas necessárias, na pasta do argumento `--directory`
- Se há espaços em branco no nome da pasta (em alguns casos pode ser incompatível com o CursorAI)

Tente também fechar e abrir o CursorAI algumas vezes para confirmar que os caminhos estão atualizados.

E quanto a outros servidores remotos?

O CursorAI aceita outros servidores MCP remotos, mesmo aqueles rodando fora do seu computador. Esse servidor pode ser seu (rodando em um servidor remoto da empresa, VPS, Cloud…) ou de terceiros.

Você encontra listagens públicas de repositórios MCP em sites como:

- https://smithery.ai/

- https://cursor.directory/
- https://mcp.pipedream.com/ Algumas dessas plataformas oferecem a hospedagem dos MCPs, de forma gratuita ou paga.

Atenção! Ao se conectar a MCPs externos, você dá permissão para que código de terceiros execute em sua máquina. **Tome cuidado e tenha certeza do que está fazendo**!

14. Miniprojeto - criando um MCP de previsão do tempo

Vamos criar um servidor de MCP conectado à API do OpenWeatherMap. O servidor terá duas tools:

- Retornar o tempo atual para um local qualquer.
- Retornar a previsão de tempo dos próximos dias para um local qualquer.

Vamos atualizar as dependências para permitir fazer requisições com `requests`:

```
# Usando pip
pip install requests

# Usando uv
uv add requests
```

Agora, crie uma conta gratuita no OpenWeatherMap e gere uma chave de API.

Adicione a chave de API ao arquivo `.env`:

```
CHAVE_API_OPENAI="sua-chave-de-api-vai-aqui"
CHAVE_API_OPENWEATHER="sua-chave-de-api-vai-aqui"
```

E agora, vamos para o código!

Nosso objetivo aqui é criar duas funções para baixar os dados de interesse e testá-las:

`servidor_previsao_tempo.py`

```
import os

import dotenv
import requests


def buscar_tempo_atual(local: str) -> dict:
    dotenv.load_dotenv()
    app_id = os.environ['CHAVE_API_OPENWEATHER']
    url = f"https://api.openweathermap.org/data/2.5/weather"
    params = {
        'q': local,
        'appid': app_id,
        'units': 'metric',
        'lang': 'pt_br',
    }
    resposta = requests.get(url=url, params=params)
    return resposta.json()


def buscar_previsao_tempo(local: str) -> dict:
    dotenv.load_dotenv()
```

```
    app_id = os.environ['CHAVE_API_OPENWEATHER']
    url = f"https://api.openweathermap.org/data/2.5/forecast"
    params = {
        'q': local,
        'appid': app_id,
        'units': 'metric',
        'lang': 'pt_br',
    }
    resposta = requests.get(url=url, params=params)
    return resposta.json()


if __name__ == "__main__":
    tempo_atual = buscar_tempo_atual('Porto Alegre')
    print(tempo_atual)
    previsao_tempo = buscar_previsao_tempo('Porto Alegre')
    print(previsao_tempo)
```

Na próxima aula, veremos como transformar este código em um servidor MCP de fato, e conectá-lo com os clientes MCP.

15. Integrando o MCP de previsão do tempo com todos os clientes

O primeiro passo é transformar o código da aula anterior em um MCP:

`servidor_previsao_tempo.py`

```
import os

import dotenv
import requests
from fastmcp import FastMCP

servidor_mcp = FastMCP('mcp-tempo')


@servidor_mcp.tool()
async def buscar_tempo_atual(local: str) -> dict:
    dotenv.load_dotenv()
    app_id = os.environ['CHAVE_API_OPENWEATHER']
    url = f"https://api.openweathermap.org/data/2.5/weather"
    params = {
        'q': local,
        'appid': app_id,
        'units': 'metric',
        'lang': 'pt_br',
    }
    resposta = requests.get(url=url, params=params)
    return resposta.json()


@servidor_mcp.tool()
async def buscar_previsao_tempo(local: str) -> dict:
    dotenv.load_dotenv()
    app_id = os.environ['CHAVE_API_OPENWEATHER']
    url = f"https://api.openweathermap.org/data/2.5/forecast"
    params = {
        'q': local,
        'appid': app_id,
        'units': 'metric',
        'lang': 'pt_br',
    }
    resposta = requests.get(url=url, params=params)
    return resposta.json()


if __name__ == "__main__":
    servidor_mcp.run(transport='stdio')
```

Agora, vamos ver como integrar este servidor MCP dos 3 clientes de MCP que usamos neste curso:

- Cliente “manual” (código Python)
- Claude Desktop

- CursorAI

Cliente “manual” (código Python)

cliente.py

```
import asyncio
import os
from pathlib import Path

import dotenv
from fastmcp import Client
from openai import OpenAI

caminho_servidor = Path(__file__).parent / "servidor_previsao_tempo.py"
cliente_mcp = Client(caminho_servidor)


async def testar_servidor(cliente, local):
    dotenv.load_dotenv()
    api_key = os.environ['CHAVE_API_OPENAI']
    async with cliente:
        resultado_atual = await cliente.call_tool("buscar_tempo_atual", arguments={'local':
↪   local})
        resultado_previsao = await cliente.call_tool("buscar_previsao_tempo",
↪   arguments={'local': local})
        mensagem_sistema = f"""
        Você é um bot que faz buscas de previsão de tempo e sintetiza a resposta.
        O usuário buscou pela previsão de tempo no seguinte local: "{local}".
        Para esta busca, você recebeu as seguintes respostas:
        - tempo atual: {resultado_atual}
        - previsão para os próximos dias: {resultado_previsao}.
        Com base nesse conteúdo, formate uma resposta amigável ao usuário.
        """
        client = OpenAI(api_key=api_key)
        response = client.responses.create(
            model="gpt-4o-mini",
            instructions=mensagem_sistema,
            input="Qual a previsão do tempo no local indicado?",
        )
        print(response.output_text)


if __name__ == '__main__':
    local = 'Porto Alegre'
    asyncio.run(testar_servidor(cliente=cliente_mcp, local=local))
```

Claude Desktop

claude_desktop_config.json

```
{
  "mcpServers": {
    "busca-wikipedia": {
      "command": "/home/USER/.local/bin/uv",
      "args": [
        "--directory",
        "/home/USER/CAMINHO_PASTA",
        "run",
        "servidor_claude_wikipedia.py"
      ],
      "description": "Um servidor MCP local que faz buscas na wikipedia e retorna o resumo de um
       ↪  artigo."
    },
    "busca-tempo": {
      "command": "/home/juliano/.local/bin/uv",
      "args": [
        "--directory",
        "/home/USER/CAMINHO_PASTA",
        "run",
        "servidor_previsao_tempo.py"
      ],
      "description": "Um servidor MCP local que faz buscas no openweathermap.org e retorna
       ↪  tempo atual e previsão do tempo para uma localidade."
    },
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/home/USER/Desktop",
        "/home/USER/Downloads"
      ]
    }
  }
}
```

CursorAI

mcp.json

```
{
  "mcpServers": {
    "busca-wikipedia-stdio": {
      "command": "/home/USER/.local/bin/uv",
      "args": [
        "--directory",
        "/home/USER/CAMINHO_PASTA",
        "run",
        "servidor_cursorai_wikipedia_stdio.py"
      ],
      "description": "Um servidor de MCP local que faz uma busca na Wikipedia e retorna um
       ↪  resumo do principal resultado."
```

```
    },
    "busca-wikipedia-sse": {
      "url": "http://localhost:8000/sse",
      "description": "Um servidor de MCP remoto que faz uma busca na Wikipedia e retorna um
       ↪  resumo do principal resultado."
    },
    "busca-tempo": {
      "command": "/home/USER/.local/bin/uv",
      "args": [
        "--directory",
        "/home/USER/CAMINHO_PASTA",
        "run",
        "servidor_previsao_tempo.py"
      ],
      "description": "Um servidor MCP local que faz buscas no openweathermap.org e retorna o
       ↪  tempo atual para uma localidade."
    }
  }
}
```

16. Fornecendo mais contexto aos seus tools

A importância dos type hints e docstrings

No contexto do MCP, os *type hints* (tipagem de dados) e as *docstrings* (strings de documentação) desempenham um papel crucial na definição e documentação das ferramentas:

- **Type hints** especificam os tipos esperados dos parâmetros de entrada e do valor de retorno da função. Isso permite que o servidor MCP gere automaticamente esquemas de entrada e saída, facilitando a validação dos dados fornecidos pelos usuários ou pelos LLMs.
- **Docstrings** fornecem uma descrição textual da função, explicando seu propósito, os parâmetros esperados e o valor retornado. Essas descrições são utilizadas pelo servidor MCP para gerar documentação acessível e compreensível para os usuários e para os modelos de linguagem.

Ao combinar *type hints* e *docstrings*, você cria uma interface clara e autoexplicativa para suas ferramentas, promovendo uma melhor integração e usabilidade.

Exemplo prático

Considere a seguinte tool que conta o número de ocorrências de uma letra em uma palavra:

```
from fastmcp import FastMCP

mcp = FastMCP("Contador de Letras")

@mcp.tool()
def contar_letras(palavra: str, letra: str) -> int:
    """
    Conta quantas vezes uma letra aparece em uma palavra.

    Args:
        palavra: A palavra na qual contar a letra.
        letra: A letra a ser contada.

    Returns:
        O número de ocorrências da letra na palavra.
    """
    return palavra.lower().count(letra.lower())
```

Neste exemplo:

- Os *type hints* (`palavra: str`, `letra: str`, `-> int`) especificam que ambos os parâmetros são strings e que a função retorna um inteiro.
- A *docstring* fornece uma descrição clara da função, explicando o propósito, os parâmetros e o valor retornado.

O servidor MCP utiliza essas informações para gerar automaticamente o esquema da ferramenta, tornando-a facilmente acessível e compreensível para os usuários e para os modelos de linguagem.

Benefícios adicionais

Além de facilitar a integração e a usabilidade, o uso adequado de *type hints* e *docstrings* oferece outros benefícios:

- Validação automática: o servidor MCP pode validar automaticamente os dados fornecidos com base nos *type hints*, reduzindo a possibilidade de erros de execução.
- Documentação gerada automaticamente: as *docstrings* podem ser utilizadas para gerar documentação automática das ferramentas, economizando tempo e esforço na criação de manuais ou guias de uso.
- Melhor manutenção: com interfaces bem definidas, é mais fácil manter e atualizar as ferramentas, além de facilitar a colaboração entre diferentes desenvolvedores.

Conclusão

Ao desenvolver ferramentas para servidores MCP, é altamente recomendável utilizar *type hints* e *docstrings* de forma consistente e detalhada. Essa prática não apenas melhora a integração com modelos de linguagem, mas também promove uma melhor experiência para os usuários humanos, facilita a manutenção do código e contribui para a criação de sistemas mais robustos e confiáveis.

17. Outros conceitos de MCPs

No decorrer do curso, explicamos em detalhes os componentes centrais de um servidor e de um cliente MCP. No entanto, o Model Context Protocol traz outros conceitos importantes.

Nesta aula, vamos explicar quatro desses conceitos: **Prompts**, **Resources**, **Roots** e **Sampling**.

Prompts: reutilizando instruções de forma padronizada

No MCP, prompts são modelos reutilizáveis de instruções que os servidores podem expor aos clientes. Eles funcionam como templates de mensagens que facilitam a interação com modelos de linguagem, permitindo que usuários ou agentes escolham rapidamente estruturas de prompts já definidas.

Por exemplo, um servidor pode disponibilizar um prompt chamado "Resumir Documento", que aceita como argumento o conteúdo de um arquivo. O cliente, por sua vez, pode apresentar esse prompt ao usuário, que seleciona o documento a ser resumido. Isso padroniza interações comuns e evita a necessidade de escrever instruções do zero a cada vez.

Os prompts são projetados para serem controlados pelo usuário, ou seja, são expostos pelos servidores com a intenção de que o usuário possa selecioná-los explicitamente para uso.

Resources: fornecendo dados contextuais aos modelos

Resources (recursos) representam qualquer tipo de dado que um servidor MCP deseja disponibilizar aos clientes. Isso pode incluir:

- Conteúdo de arquivos
- Registros de banco de dados
- Respostas de APIs
- Dados de sistemas ao vivo
- Capturas de tela e imagens
- Arquivos de log
- E mais

Cada recurso é identificado por um URI único e pode conter dados em texto ou binários .

Esses recursos fornecem o contexto necessário para que os modelos de linguagem compreendam melhor as tarefas que precisam realizar, permitindo respostas mais precisas e relevantes.

Roots: delimitando o escopo de atuação dos servidores

Roots são um conceito no MCP que define os limites onde os servidores podem operar. Eles fornecem uma maneira para os clientes informarem aos servidores sobre os recursos relevantes e suas localizações.

Por exemplo, um cliente pode definir uma pasta específica como root, permitindo que o servidor MCP acesse apenas os arquivos dentro dessa pasta. Isso garante que o servidor opere dentro de um escopo bem definido, aumentando a segurança e a organização.

Além disso, múltiplos roots podem ser utilizados simultaneamente, permitindo que o servidor trabalhe com diferentes conjuntos de recursos conforme necessário.

Sampling: solicitando completions de LLMs via cliente

Sampling é um recurso poderoso do MCP que permite que os servidores solicitem completions de modelos de linguagem (LLMs) por meio do cliente, habilitando comportamentos agentivos sofisticados enquanto mantêm a segurança e a privacidade.

O fluxo de sampling segue estas etapas:

1. O servidor envia uma solicitação `sampling/createMessage` para o cliente.
2. O cliente revisa a solicitação e pode modificá-la.
3. O cliente realiza o sampling com um LLM.
4. O cliente revisa a completion.
5. O cliente retorna o resultado ao servidor.

Esse design com intervenção humana garante que os usuários mantenham o controle sobre o que o LLM vê e gera.

Conclusão

Esses quatro conceitos - Prompts, Resources, Roots e Sampling - expandem significativamente as capacidades do MCP, permitindo interações mais ricas, seguras e organizadas entre servidores e clientes. Compreender e utilizar esses recursos pode aprimorar a forma como você desenvolve e integra soluções baseadas em modelos de linguagem.

18. Encerramento

Olá! Chegamos ao fim do nosso curso de MCPs. Foi uma jornada incrível: começamos entendendo aplicações simples de IA, vimos como usar agents e tools, aprendemos a criar servidores e clientes MCP, e descobrimos como padronizar integrações complexas.

Recapitulação dos aprendizados

- No Módulo 1, você conheceu a arquitetura básica de uma aplicação IA e como Agents e Tools funcionam.
- No Módulo 2, colocou a mão na massa criando servidores MCP locais e remotos em Python.
- No Módulo 3, construiu um cliente manual para entender de fato o que faz um cliente MCP.
- No Módulo 4, conectou seus servidores a clientes como Claude Desktop e CursorAI.
- No Módulo 5, explorou o valor do MCP na padronização de ferramentas e na criação de um ecossistema de "plugins".

Próximos passos

- **Experimente**: publique seu servidor MCP de previsão do tempo ou crie um novo plugin para seu caso de uso.
- **Compartilhe**: contribua com a comunidade MCP abrindo um repositório ou criando uma integração.
- **Aprofunde**: revise a especificação oficial em modelcontextprotocol.io e acompanhe as novidades no GitHub.

Dicas para ir além

- Pense em cenários reais na sua empresa ou projeto pessoal onde várias ferramentas precisam conversar de forma padronizada.
- Teste diferentes clientes MCP (Python, JavaScript, desktop) para ver qual se encaixa melhor no seu fluxo.
- Fique de olho em novas atualizações do protocolo e em projetos inspiradores da comunidade.

Até seu próximo curso aqui na Asimov Academy!